NeOS MiniOS Final Project

I. Introduction

MiniOS is a simple operating system simulation made using Python. It shows how an operating system works by providing basic features such as file management, process management, memory management, user management, and system information. Users can interact with MiniOS by typing commands into the command line.

II. Project Objectives

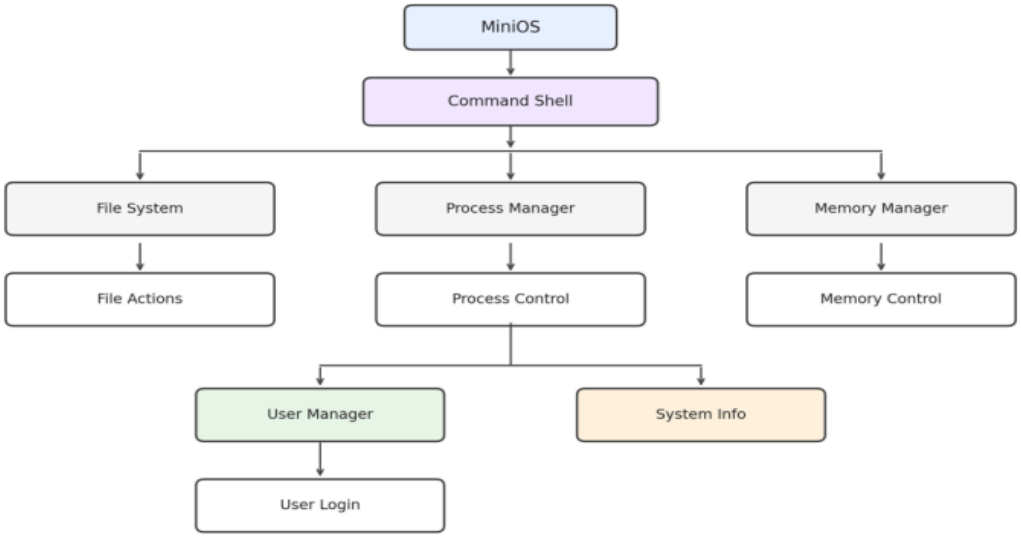
The main goal of MiniOS is to show how an operating system works using Python.

The objectives of the project are:

- To create a command line where users can enter commands.
- To allow users to create, view, edit, and delete files.
- To show how processes are created and managed.
- To simulate how memory is used and managed.
- To create a simple login system for users.
- To display basic information about the system.
- To improve the group's Python programming and problem-solving skills.

III. System Architecture

The system architecture shows how the different parts of MiniOS work together. Users enter commands through the command shell, which sends the commands to the correct part of the system.



MiniOS System Architecture

Description of Each Component
Command Shell: Receives commands from users and sends them to the correct part of the system.
File System: Handles file operations, such as creating, viewing, editing, and deleting files.
Process Manager: Handles the creation, tracking, and stopping of processes.
Memory Manager: Tracks how much memory is being used and how much is still available.
User Manager: Handles user accounts and checks login information.
System Information: Shows basic details about MiniOS and its resources.

IV. Program Structure

Program Structure

NeOS Mini OS is made using Python. The program can be divided into different files so that each file handles a specific task.

Here is an example of how the files can be organized:

```
NeOS/
│
├── main.py
├── filesystem.py
├── processes.py
├── memory.py
├── users.py
```

Description of Python Files

main.py – Starts the Mini OS program and prepares the parts needed to run it.

filesystem.py – Handles file operations, such as creating, reading, editing, listing, and deleting files.

processes.py – Manages the simulated processes in the system.

memory.py – Keeps track of used and available memory.

users.py – Handles user login and account information.

V. File Management

File Management allows users to create, view, edit, list, and delete files through commands. It helps organize and manage files by storing their names and contents in the system.

VI. Process Management

Process Management simulates how an operating system handles running tasks. Users can create, view, and stop processes. The system tracks each process and updates its status when changes occur.

VII. Memory Management

Memory Management simulates how the system allocates and releases memory. It tracks the amount of memory used and the remaining available memory. When a process is created or stopped, the memory usage is updated accordingly.

VIII. User Management

User Management handles user accounts and login authentication. Users enter their usernames and passwords, and the system checks whether the credentials are correct. If the information is valid, the user is allowed to log in; otherwise, access is
denied.

IX. Error Handling

Error Handling helps MiniOS manage invalid commands and unexpected problems. When users enter an unknown command or provide incorrect information, the system displays an error message instead of stopping the program. It also checks user inputs to prevent invalid operations, such as accessing a file that does not exist or entering incorrect login credentials. This makes the system more reliable and easier to use.

X. Testing

The group tested MiniOS to check if its features were working properly. The following table shows the test cases and their expected results.

No.	Test Case	Action	Expected Result
1	User Login	Enter the correct username and password.	The user can log in successfully.
2	Incorrect Login	Enter the wrong username or password.	An error message appears.
3	Create File	Enter a command to create a file.	A new file is created.
4	Read File	Open an existing file.	The file contents are displayed.
5	Delete File	Delete an existing file.	The file is deleted.
6	Invalid Command	Enter an unknown command.	An error message appears.
7	Create Process	Enter a command to create a process.	A new process is added to the process list.

8	Stop Process	Enter a valid process ID to stop it.	The selected process is stopped or removed from the active process list.
9	Allocate Memory	Request an amount of available memory.	The requested memory is allocated.
10	Insufficient Memory	Request more memory than is available.	The system rejects the request.
11	System Information	Enter the system information command.	The system displays its information.

XI. Problems Encountered

While developing MiniOS, the group may have faced problems when writing the code, connecting the different files, and making sure that each feature worked properly.

Some possible problems and solutions are:

Invalid Commands: The program may not recognize commands entered by users. To solve this, the group can add command checking and display an error message for unknown commands.

File Errors: Users may try to open or delete files that do not exist. The group can solve this by checking whether a file exists before performing the action.

Memory Errors: The amount of available memory may become incorrect after several operations. The group can solve this by updating the memory values after each successful allocation or release.

Process Errors: It may be difficult to identify processes without unique IDs. The group can solve this by giving each process its own ID.

Problems Connecting Python Files: Some parts of the program may not work together because of incorrect imports or function names. The group can solve this by checking the code and making sure that the files are connected correctly.

These problems can be solved through debugging, testing, and checking the code carefully.

XII. Conclusion

The MiniOS project helped the group learn about the basic functions of an operating system. By using Python, the group was able to simulate file management, process management, memory management, user login, and system information.

The project also helped the group improve its programming, debugging, testing, and problem-solving skills. Although MiniOS is not a real operating system, it helps explain how an operating system manages different tasks and resources.